Five-Drone Leader–Follower Formation and Dynamic Multi-Drone Area Survey with Task Reallocation

ROS 2 Humble · AeroStack2 · Gazebo · DDS

1. Executive Summary

Five-Drone Leader–Follower Formation and Dynamic Multi-Drone Area Survey with Task Reallocation project delivers a five-drone leader–follower coordination system, with formation control and dynamic area-survey task reallocation, developed and demonstrated entirely in simulation on ROS 2 Humble, AeroStack2 and Gazebo. Drone0 is the leader; drone1, drone2, drone3 and drone4 are followers.

The **AeroStack2 command-based implementation** uses the DroneInterface abstraction for high-level behaviours (arm, offboard, takeoff, go-to, land).

The **direct formation implementation** closes the loop itself, using ground-truth pose feedback and Gazebo velocity topics.

The **DDS implementation** separates leader command publication from follower execution, so each follower runs as an independent process.

The **task-allocation implementation** extends the system from formation movement to independent survey assignments, waypoint status reporting and reassignment of unfinished work.

In the advanced task workflow, area A is allocated to drone1, area B to drone2 and area C to drone3, while drone4 is held as a helper. In the programmed delay scenario, drone1 reports its remaining waypoints; the leader transfers those waypoints to drone4 and places drone1 in HOLD.

1.1 Delivered Capabilities

- Five-drone simulation with a defined leader and four followers.
- Line and V/arrow formation control paths.
- AeroStack2 high-level mission-control implementation.
- Direct feedback-based Gazebo velocity control.
- DDS-based command and leader-pose distribution.
- Individual survey-area assignment and waypoint progress reporting.
- Programmed delay handling with transfer of unfinished waypoints to an available follower.

1.2 System Configuration

Item	Configuration
Leader	drone0
Followers	drone1, drone2, drone3, drone4
Operating system	Ubuntu 22.04
Middleware	ROS 2 Humble
Aerial robotics framework	AeroStack2
Simulator	Gazebo / Ignition Gazebo
Communication	ROS 2 topics over DDS
Programming	Python 3 / rclpy / Bash

2. Problem Definition

A multi-drone mission needs coordination at several levels: vehicle-level movement, group-level formation geometry, inter-process communication, individual task execution, and mission-level recovery when a follower is delayed. The project therefore separates simulation and vehicle interfaces, motion control, leader–follower messaging, task execution, and task supervision/reallocation.

Functional Requirements

- Represent five simulated vehicles using namespaces drone0 through drone4.
- Assign drone0 the leader role and the remaining vehicles follower roles.
- Support takeoff, landing, line formation and V/arrow formation.
- Exchange formation commands and leader position information through ROS 2 topics.
- Assign different survey waypoint sets to individual followers.
- Receive and interpret follower task-status messages.
- Reassign unfinished waypoints in the programmed delay scenario.

Layer	Responsibility	Implementation
Simulation	Vehicle models, physics and Gazebo topics	Gazebo / Ignition
Vehicle abstraction	High-level arming, offboard, takeoff, GoTo and landing	AeroStack2 DroneInterface
Motion control	Position feedback and velocity generation	direct_formation.py / DDS program
Coordination	Leader commands and pose distribution	leader_follower_dds.py
Task execution	Individual survey tasks and waypoint tracking	leader_tasks.py

3. Software and Simulation Environment

The project runs on Ubuntu 22.04 with ROS 2 Humble and an AeroStack2 workspace, using Gazebo / Ignition Gazebo as the simulator.

Component	Configuration / use
Ubuntu	22.04 development and runtime environment
ROS 2	Humble distribution
AeroStack2 workspace	~/aerostack2_ws
Project root	~/project_gazebo
Gazebo world configuration	config/world_swarm_5drone.yaml
Simulation launcher	launch_as2_5drone.bash
Python ROS client	rclpy
Transport	DDS / Fast DDS profile
Simulation control	Gazebo / Ignition Transport topics

3.1 Environment Initialisation

The environment loads ROS 2 and AeroStack2, points Fast DDS at a UDP-only profile, and disables the ROS 2 CLI daemon so that command-line checks reflect live state.

```
source /opt/ros/humble/setup.bash
source ~/aerostack2_ws/install/setup.bash
export
FASTRTPS_DEFAULT_PROFILES_FILE="$HOME/project_gazebo/config/fastdds_udp_only.xml"
export
FASTDDS_DEFAULT_PROFILES_FILE="$HOME/project_gazebo/config/fastdds_udp_only.xml"
export ROS2CLI_DISABLE_DAEMON=1
cd ~/project_gazebo
```

3.2 Five-Drone Simulation Launch

The `-m` option selects multi-agent mode. The launcher selects the five-drone world configuration, creates the vehicle sessions and starts the corresponding simulation and AeroStack2 components.

```
cd ~/project_gazebo
./launch_as2_5drone.bash -m
```

3.3 Session Visibility

The launcher creates one tmux session per vehicle. The expected sessions are drone0, drone1, drone2, drone3 and drone4.

```
tmux list-sessions
```

4. Architecture and Communication

The leader never drives followers through shared memory or direct calls; every interaction is a ROS 2 topic over DDS, so each drone can run as an independent process and be inspected on its own.

4.1 Fleet Roles

Drone	Role	Assigned function
drone0	Leader	Formation command, leader pose, task allocation and reassignment
drone1	Follower 1	Formation follower; initial survey area A
drone2	Follower 2	Formation follower; initial survey area B
drone3	Follower 3	Formation follower; initial survey area C
drone4	Follower 4	Formation follower; reassignment support

4.2 Communication Interfaces

Topic	Message type	Function
/formation/command	std_msgs/String	Leader broadcasts formation or flight command
/formation/leader_pose	geometry_msgs/PoseStamped	Leader publishes pose reference
/formation/task/droneN	std_msgs/String	Leader sends an individual task to follower N
/formation/status	std_msgs/String	Follower reports progress and task status
/droneN/ground_truth/pose	geometry_msgs/PoseStamped	Ground-truth position feedback
/gz/droneN/cmd_vel	geometry_msgs/Twist	Velocity command interface used by direct-control nodes

4.3 Program Variants and Their Boundaries

File	Approach	What it demonstrates
leader_follower_command_demo.py	AeroStack2 DroneInterface	High-level vehicle behaviour orchestration
direct_formation.py	Pose feedback + Gazebo velocity	Centralised direct formation controller
leader_follower_dds.py	Leader/follower ROS 2 processes	Distributed formation command execution

File	Approach	What it demonstrates
leader_tasks.py	Individual task topics + status topic	Distributed task execution and leader reassignment
run_leader_follower.sh	Bash	DDS formation process startup
run_leader_tasks.sh	Bash	Task-management process startup

5. Formation Control Design

Formation is expressed as a fixed offset from the leader, and each follower converges on its target with a clipped proportional controller. This is deliberately simple: it keeps the cause of any error traceable to either the offset geometry or the controller gain.

5.1 Relative-Offset Model

Each follower has a fixed (x, y) offset for the selected formation mode. The follower target is the leader reference position plus that offset, held at the common target altitude.

5.2 Line Formation

Vehicle	Offset (x, y), m
drone0	(0, 0)
drone1	(-1.5, 0)
drone2	(-3.0, 0)
drone3	(-4.5, 0)
drone4	(-6.0, 0)

5.3 V / Arrow Formation

Vehicle	Offset (x, y), m
drone0	(0, 0)
drone1	(-1.5, 1.5)
drone2	(-1.5, -1.5)
drone3	(-3.0, 3.0)
drone4	(-3.0, -3.0)

5.4 Direct Position Feedback

The direct-control implementation subscribes to each vehicle's ground-truth pose and publishes Twist commands to the corresponding Gazebo velocity topic. It computes the position error between target

and current position, applies a proportional gain and limits the velocity command. The vertical term is scaled by 2 so altitude converges faster than horizontal position.

```
e_x = x_{target} - x,\quad e_y = y_{target} - y,\quad e_z = z_{target} - z
v_x = \mathrm{clip}(K_P\,e_x,\ V_{XY}),\quad v_y = \mathrm{clip}(K_P\,e_y,\ V_{XY})
v_z = \mathrm{clip}(2K_P\,e_z,\ V_{UP}\ \text{or}\ V_{DN})
```

Parameter	Value in direct_formation.py
Target altitude	1.05 m
Ground reference	0.06 m
Proportional gain KP	0.8
Horizontal velocity limit	0.6 m/s
Upward velocity limit	0.5 m/s
Downward velocity limit	0.4 m/s
Position tolerance	0.15 m

The velocity limits matter as much as the gain. A proportional term alone would command large velocities for large errors; clipping keeps the response bounded and the descent slower than the climb.

5.5 AeroStack2 Behaviour Interface

The command demo uses AeroStack2 DroneInterface to access vehicle-level behaviours, so the mission layer is expressed as arming, offboard mode, takeoff, GoTo and landing rather than raw Gazebo velocity messages. In this variant the formation is command-driven: the leader is held at its nominal reference point and does not continuously track the leader's measured position.

5.6 DDS Leader–Follower Interface

The DDS formation implementation starts one process per follower and a separate leader process. The leader publishes a command and its own pose; each follower receives both and computes its own target from its offset. Unlike the command demo, followers here track the leader's live position.

The recorded demonstration command sequence was T (TAKEOFF), 1 (LINE), 2 (ARROW), 1 (LINE), and L (LAND).

6. Dynamic Area Survey and Task Reallocation

The leader assigns each follower its own survey area, tracks progress through a shared status topic, and moves unfinished waypoints to an idle helper when a follower reports a delay.

6.1 Survey Geometry

Three survey areas are defined, each as four planar waypoints.

Area	Initial owner	Waypoints (x, y), m
A	drone1	(-2.5, -2.0), (-2.5, 2.0), (-1.0, 2.0), (-1.0, -2.0)
B	drone2	(-0.5, -2.0), (-0.5, 2.0), (0.5, 2.0), (0.5, -2.0)
C	drone3	(1.0, -2.0), (1.0, 2.0), (2.5, 2.0), (2.5, -2.0)
Helper	drone4	Available for reassignment in the programmed delay scenario

Each drone also flies at its own altitude: drone1 at 1.05 m, with a further 0.3 m added per drone index (drone2 1.35 m, drone3 1.65 m, drone4 1.95 m), so that vehicles do not share a flight level during the survey.

6.2 Individual Task Messaging

The leader creates one publisher per follower task topic. Each follower subscribes only to its own task topic, parses the JSON payload and executes the task. The shared status topic lets the leader follow progress without controlling every waypoint transition.

6.3 Task Types

Task	Purpose
TAKEOFF	Command a follower to take off
LAND	Command a follower to land
HOLD	Stop task progression and hold
LINE	Adopt line formation
ARROW	Adopt V/arrow formation
GOTO	Move to a specified target
SURVEY	Execute a specified survey waypoint list

6.4 Delay-Triggered Reallocation

1. The leader assigns survey area A to drone1, area B to drone2 and area C to drone3.
2. Drone1 completes its initial waypoints and reports a delay with its remaining waypoint indices.
3. The leader identifies drone4 as an available helper.
4. The leader transfers the unfinished area A waypoints to drone4.
5. The leader sends HOLD to drone1.
6. Drone4 executes the transferred waypoints and reports completion.

The recorded output shows drone1 completing A1 and A2, then reporting remaining waypoints A3 and A4. The leader transferred those waypoints to drone4, which subsequently reported completion of A3

and A4. A separate captured run showed areas A and B at 4/4 while drone3 was still active on area C at the time of that status snapshot.

6.5 Why This Design Matters

The architecture separates the decision to assign work from the process that executes it. The leader can issue work to one follower without broadcasting it to the whole fleet, and status reporting gives it the information needed to update the mission view and transfer unfinished work when the programmed delay condition occurs.

7. Engineering Diagnosis and Integration

Every integration problem was diagnosed by asking which layer the evidence pointed to, rather than by changing several things at once.

7.1 Model and Plugin Compatibility

The Gazebo control plugin configuration was checked against the installed simulator version. The Fortress-compatible identifier ignition-gazebo-multicopter-control-system was found to load and respond, while the gz-sim-prefixed identifier used in an earlier configuration did not exist in that environment.

7.2 Spawn and Startup Sequencing

Multi-drone startup required separate checks for model creation and controller availability. The manual ros_gz_sim create path was used as a diagnostic and recovery method to determine whether the fault lay in model spawning or in a later control layer. It is distinct from the normal five-drone launcher.

7.3 Command-to-Response Diagnosis

The control path was examined in stages: confirm the ROS publisher, inspect the bridge and Gazebo topic, check whether the velocity controller was enabled, then observe the Gazebo pose. Each stage isolates a failure to the command publisher, transport bridge, controller activation or vehicle response.

7.4 Simulation-Time Awareness

The Gazebo real-time factor was observed during diagnosis. When it is low, a duration measured on the wall clock does not correspond to the same duration in simulation time. This mattered when interpreting takeoff and movement timing.

7.5 Monitoring and Diagnostic Commands

```
ign model --list
ign topic -l
ign topic -i -t /model/drone0/cmd_vel
ign topic -e -n 1 -t /model/drone0/pose
ros2 topic echo /drone0/ground_truth/pose --once
ros2 topic echo /drone0/platform/info --once
tmux list-sessions
```

Fleet-wide altitude and arming/offboard checks:

```
for d in 0 1 2 3 4; do
    echo "drone$d"
    timeout 8 ros2 topic echo /drone$d/ground_truth/pose --once | grep -A3 "position"
done

for d in 0 1 2 3 4; do
    echo "--- drone$d ---"
    timeout 8 ros2 topic echo /drone$d/platform/info --once | grep -E
"armed|offboard"
done
```

7.6 Manual Controller Check

```
ign topic -t /model/drone0/velocity_controller/enable -m ignition.msgs.Boolean -p
'data: true'
ign topic -t /model/drone0/cmd_vel -m ignition.msgs.Twist -p 'linear: {z: 0.5}'
ign topic -t /model/drone0/cmd_vel -m ignition.msgs.Twist -p 'linear: {z: 0.0}'
```

These are diagnostic checks. They are not a replacement for the mission programs or a physical flight procedure.

7.7 Manual Model Spawning

```
ros2 run ros_gz_sim create -world empty -file /tmp/quadrotor_base_0.sdf -x 0 -y 0 -z
0.2

for i in 1 2 3 4; do
    case $i in
        1) P="-x -2 -y -1.5" ;;
        2) P="-x -2 -y 1.5" ;;
        3) P="-x 0 -y -3" ;;
        4) P="-x 0 -y 3" ;;
    esac
    ros2 run ros_gz_sim create -world empty -file /tmp/quadrotor_base_$i.sdf $P -z
0.2
done
```

The generated model files and plugin were validated with:

```
for i in 0 1 2 3 4; do
    python3 -c "
import xml.dom.minidom as m
m.parse('/tmp/quadrotor_base_$i.sdf')
print('$i XML OK')
"
done

grep -n -E "multicopter-control|MulticopterVelocityControl" /tmp/quadrotor_base_0.sdf
```

8. Verification and Recorded Results

The project owner has confirmed that the project was completed and verified in simulation; the maximum position errors were 0.03 m at takeoff, 0.04 m in line formation, 0.02 m in V formation and 0.05 m at landing. These figures come from the project execution notes and are recorded simulation-run observations.

8.1 Recorded Maximum Position Error

Operation	Recorded maximum position error
Takeoff	0.03 m
Line formation	0.04 m
V formation	0.02 m
Landing	0.05 m

8.2 Functional Verification Matrix

Test area	Expected behaviour	Reported project outcome
Fleet initialisation	Five drone interfaces/processes available	Five-drone project completed and verified
Takeoff / landing	All five vehicles execute lifecycle commands	Included in formation demonstration
Line formation	Followers adopt line offsets	Demonstrated
V / arrow formation	Followers adopt V offsets	Demonstrated
DDS command path	Followers respond to leader command sequence	Demonstrated
Survey allocation	A/B/C assigned to individual followers	Implemented and demonstrated
Delay handling	Remaining A waypoints transferred to drone4	Recorded output confirms A3/A4 completion by drone4

8.3 Interpretation of Results

The formation errors are the maxima captured for each simulation operation. They must not be read as hardware flight accuracy or a statistical performance guarantee. The reallocation output demonstrates the programmed recovery path; it does not establish a general reliability percentage or a reassignment latency.

8.4 Recorded Task Output

```
[LEADER] drone1 finished A1
[LEADER] drone1 finished A2
[LEADER] drone1 reports DELAY, remaining A waypoints [3, 4]
[LEADER] TRANSFER: drone1 -> drone4, area A
[LEADER] -> drone4: SURVEY area A waypoints [3, 4]
[LEADER] -> drone1: HOLD
[LEADER] drone4 finished A3
```

```
[LEADER] drone4 finished A4
[LEADER] drone4 finished its task
```

8.5 Evidence to Retain for Handover

- Final Gazebo demonstration recording or screenshots.
- Leader terminal output for the formation command sequence.
- Follower logs from /tmp/follower1.log through /tmp/follower4.log.
- Task leader output showing DELAYED, TRANSFER and WP_DONE messages.
- ROS topic snapshots for ground-truth pose and platform status.

9. Execution and Operations

Each program is run separately; start the simulation first, then exactly one control variant at a time.

9.1 Launch the Simulation

```
cd ~/project_gazebo
./launch_as2_5drone.bash -m
```

9.2 Run the AeroStack2 Command Demo

```
python3 leader_follower_command_demo.py
```

Commands at the prompt: TAKEOFF, LINE, V_FORMATION, LAND and EXIT.

9.3 Run the Direct Formation Controller

```
python3 direct_formation.py
```

Commands at the prompt: TAKEOFF, LINE, V_FORMATION, LAND and EXIT.

9.4 Run DDS Formation

```
chmod +x run_leader_follower.sh
./run_leader_follower.sh
```

Leader commands: 1 = LINE, 2 = ARROW, T = TAKEOFF, L = LAND, `g dx dy` = move leader, q = quit.

9.5 Run Task Allocation

The run_leader_tasks.sh launcher was created from the previous launcher by substituting the script name:

```
sed 's/leader_follower_dds.py/leader_tasks.py/g' \
    ~/project_gazebo/run_leader_follower.sh \
    > ~/project_gazebo/run_leader_tasks.sh

chmod +x ~/project_gazebo/run_leader_tasks.sh
./run_leader_tasks.sh
```

Leader commands: T = takeoff, L = land, 1 = LINE, 2 = ARROW, `mission [delay]`, `assign N HOLD|LAND|LINE|ARROW|GOTO x y z`, `g dx dy`, `status`, `q`. Use `mission delay` to demonstrate survey allocation with reassignment.

9.6 Syntax Validation

```
python3 -m py_compile leader_follower_command_demo.py
python3 -m py_compile direct_formation.py
python3 -m py_compile leader_follower_dds.py
python3 -m py_compile leader_tasks.py
```

9.7 Inspecting Follower Logs

```
cat /tmp/follower1.log
cat /tmp/follower2.log
cat /tmp/follower3.log
cat /tmp/follower4.log
```

9.8 Stop the Simulation

Ensure the active simulated drones have received a landing command first. `tmux kill-server` was used only for a complete session reset.

```
cd ~/project_gazebo
./stop.bash
```

10. Conclusion

The project integrates five-drone simulation, leader–follower formation control and dynamic area-survey task reallocation using ROS 2 Humble, AeroStack2 and Gazebo. It includes multiple control paths: high-level vehicle behaviours, direct feedback-based motion, and distributed leader–follower communication.

The task-management layer extends the system from formation movement to individual work allocation. Followers receive separate survey tasks, report waypoint progress and support a programmed recovery case in which unfinished waypoints move from a delayed follower to an available helper.

Appendix 1 – leader_follower_command_demo.py

Complete source listing reproduced from the supplied project archive. No source-code lines have been intentionally omitted.

```
#!/usr/bin/env python3

import sys
import time

import rclpy
from as2_python_api.drone_interface import DroneInterface


# CONFIGURATION
DRONE_NAMES = [
    "drone0",
    "drone1",
    "drone2",
    "drone3",
    "drone4",
]

LEADER = "drone0"

TAKEOFF_HEIGHT = 1.0
TAKEOFF_SPEED = 0.5
MISSION_SPEED = 0.6
LAND_SPEED = 0.4

FRAME_ID = "earth"
STABILIZATION_TIME = 3.0
FORMATION_SPACING = 1.5

# Leader's nominal formation reference position.
LEADER_X = 0.0
LEADER_Y = 0.0
FLIGHT_HEIGHT = TAKEOFF_HEIGHT


# FORMATION OFFSETS
# Positions are relative to the leader's nominal
# reference position, using the earth frame.
# LINE: followers form a line behind the leader.
LINE_OFFSETS = {
    "drone1": (-FORMATION_SPACING, 0.0),
    "drone2": (-2 * FORMATION_SPACING, 0.0),
    "drone3": (-3 * FORMATION_SPACING, 0.0),
    "drone4": (-4 * FORMATION_SPACING, 0.0),
}

# V formation: two drones on each side.
V_OFFSETS = {
    "drone1": (-FORMATION_SPACING, FORMATION_SPACING),
    "drone2": (-FORMATION_SPACING, -FORMATION_SPACING),
```

```
    "drone3": (-2 * FORMATION_SPACING, 2 * FORMATION_SPACING),
    "drone4": (-2 * FORMATION_SPACING, -2 * FORMATION_SPACING),
}


# DRONE OPERATIONS
def initialize_drones():
    """Create the five AeroStack2 drone interfaces."""
    drones = {}

    for name in DRONE_NAMES:
        print(f"[INIT] Connecting to {name}...")

        drones[name] = DroneInterface(
            name,
            verbose=True,
            use_sim_time=True,
        )


    print("[INIT] All five interfaces initialized.")
    return drones


def takeoff_all(drones):
    """Arm, enter offboard mode, and take off."""
    print("\n[TAKEOFF] Preparing all five drones.")

    # Arm all drones first.
    for name in DRONE_NAMES:
        print(f"[ARM] {name}")
        drones[name].arm()

    # Enter offboard mode.
    for name in DRONE_NAMES:
        print(f"[OFFBOARD] {name}")
        drones[name].offboard()

    # Take off one at a time, waiting for each
    # blocking takeoff operation to complete.
    for name in DRONE_NAMES:
        print(f"[TAKEOFF] {name}")
        drones[name].takeoff(
            TAKEOFF_HEIGHT,
            TAKEOFF_SPEED,
            True,
        )

    print(f"[TAKEOFF] Waiting {STABILIZATION_TIME} seconds.")
    time.sleep(STABILIZATION_TIME)

    print("[TAKEOFF] Sequence completed.")
```

```
def move_to_formation(drones, formation_name):
    if formation_name == "LINE":
        offsets = LINE_OFFSETS
    elif formation_name == "V_FORMATION":
        offsets = V_OFFSETS
    else:
        raise ValueError(f"Unknown formation: {formation_name}")

    print(f"\n[LEADER COMMAND] {LEADER} selected {formation_name}")

    # Hold the leader at its nominal reference point.
    print(f"[LEADER] Moving to reference position.")
    drones[LEADER].go_to.go_to(
        LEADER_X,
        LEADER_Y,
        FLIGHT_HEIGHT,
        MISSION_SPEED,
        FRAME_ID,
    )

    # Command each follower to its formation position.
    for name in DRONE_NAMES:
        if name == LEADER:
            continue

        offset_x, offset_y = offsets[name]

        target_x = LEADER_X + offset_x
        target_y = LEADER_Y + offset_y

        print(
            f"[FOLLOWER] {name} -> "
            f"x={target_x:.2f}, "
            f"y={target_y:.2f}, "
            f"z={FLIGHT_HEIGHT:.2f}"
        )

        drones[name].go_to.go_to(
            target_x,
            target_y,
            FLIGHT_HEIGHT,
            MISSION_SPEED,
            FRAME_ID,
        )

    print(f"[FORMATION] {formation_name} command completed.")


def land_all(drones):
    """Request landing for all five drones."""
    print("\n[LAND] Starting landing sequence.")

    for name in DRONE_NAMES:
        print(f"[LAND] {name}")
        drones[name].land(LAND_SPEED, True)
```

```
    print("[LAND] Landing sequence completed.")


# COMMAND LOOP
def main():
    rclpy.init()

    drones = {}
    airborne = False

    print("=" * 55)
    print(" FIVE-DRONE LEADER-FOLLOWER FORMATION DEMO")
    print("=" * 55)
    print(f"Leader: {LEADER}")
    print(f"Followers: {', '.join(DRONE_NAMES[1:])}")
    print()

    try:
        drones = initialize_drones()

        print("\nAvailable commands:")
        print("  TAKEOFF      - Take off all five drones")
        print("  LINE         - Switch to line formation")
        print("  V_FORMATION  - Switch to V formation")
        print("  LAND         - Land all five drones")
        print("  EXIT         - Exit the demo")
        print()

        while True:
            command = input("Leader command > ").strip().upper()

            if command == "TAKEOFF":
                if airborne:
                    print("[INFO] Drones are already marked airborne.")
                    continue

                takeoff_all(drones)
                airborne = True

            elif command in ("LINE", "V_FORMATION"):
                if not airborne:
                    print("[WARNING] Enter TAKEOFF first.")
                    continue

                move_to_formation(drones, command)

            elif command == "LAND":
                if not airborne:
                    print("[INFO] The takeoff sequence has not run.")
                    continue

                land_all(drones)
                airborne = False
```

```
            elif command == "EXIT":
                if airborne:
                    print(
                        "[SAFETY] Drones are marked airborne. "
                        "Use LAND before EXIT."
                    )
                    continue

                break

            elif command == "":
                continue

            else:
                print("[ERROR] Unknown command.")

    except KeyboardInterrupt:
        print(
            "\n[INTERRUPTED] Check the drones' actual state "
            "in Gazebo and land safely if necessary."
        )

    except Exception as exc:
        print(f"\n[ERROR] Demo stopped: {exc}")
        print(
            "[SAFETY] Check the actual state of every drone "
            "before issuing further commands."
        )
        raise

    finally:
        for name, drone in drones.items():
            try:
                drone.shutdown()
            except Exception:
                pass

        if rclpy.ok():
            rclpy.shutdown()


if __name__ == "__main__":
    main()
```

Appendix 2 – direct_formation.py

Complete source listing reproduced from the supplied project archive. No source-code lines have been intentionally omitted.

```
#!/usr/bin/env python3
import time, rclpy
from rclpy.node import Node
from rclpy.qos import qos_profile_sensor_data
from geometry_msgs.msg import Twist, PoseStamped
```

```
N = 5
LINE = [(0,0),(-1.5,0),(-3.0,0),(-4.5,0),(-6.0,0)]
V    = [(0,0),(-1.5,1.5),(-1.5,-1.5),(-3.0,3.0),(-3.0,-3.0)]
ALT, GROUND = 1.05, 0.06
KP, VXY, VUP, VDN, TOL = 0.8, 0.6, 0.5, 0.4, 0.15

class Fleet(Node):
    def __init__(self):
        super().__init__('direct_formation')
        self.pos = [None]*N
        self.pub = []
        for i in range(N):
            self.create_subscription(PoseStamped, f'/drone{i}/ground_truth/pose',
                lambda m, i=i: self.pos.__setitem__(i, (m.pose.position.x,
m.pose.position.y, m.pose.position.z)),
                qos_profile_sensor_data)
            self.pub.append(self.create_publisher(Twist, f'/gz/drone{i}/cmd_vel',
10))
        while None in self.pos:
            rclpy.spin_once(self, timeout_sec=0.2)
        self.origin = self.pos[0][:2]

    @staticmethod
    def clip(v, lim): return max(-lim, min(lim, v))

    def goto(self, targets, vup=VUP, vdn=VDN, timeout=240, hold=2.0):
        ok_since, t0 = None, time.time()
        while time.time() - t0 < timeout:
            rclpy.spin_once(self, timeout_sec=0.05)
            worst = 0.0
            for i, (tx, ty, tz) in enumerate(targets):
                x, y, z = self.pos[i]
                ex, ey, ez = tx-x, ty-y, tz-z
                worst = max(worst, abs(ex), abs(ey), abs(ez))
                m = Twist()
                m.linear.x = self.clip(KP*ex, VXY)
                m.linear.y = self.clip(KP*ey, VXY)
                m.linear.z = self.clip(KP*ez*2, vup if ez > 0 else vdn)
                self.pub[i].publish(m)
            if worst < TOL:
                ok_since = ok_since or time.time()
                if time.time() - ok_since > hold: break
            else:
                ok_since = None
        print('  max error %.2f m' % worst)

    def formation(self, offs):
        ox, oy = self.origin
        self.goto([(ox+dx, oy+dy, ALT) for dx, dy in offs])

    def stop(self):
        for p in self.pub: p.publish(Twist())

def main():
    rclpy.init()
```

```
    f = Fleet()
    print('Commands: TAKEOFF LINE V_FORMATION LAND EXIT')
    while True:
        c = input('Leader command > ').strip().upper()
        if c == 'TAKEOFF':
            f.goto([(f.pos[i][0], f.pos[i][1], ALT) for i in range(N)])
        elif c == 'LINE': f.formation(LINE)
        elif c == 'V_FORMATION': f.formation(V)
        elif c == 'LAND':
            f.goto([(f.pos[i][0], f.pos[i][1], GROUND) for i in range(N)])
        elif c == 'EXIT': break
        f.stop()
    rclpy.shutdown()
main()
```

Appendix 3 – leader_follower_dds.py

Complete source listing reproduced from the supplied project archive. No source-code lines have been intentionally omitted.

```
#!/usr/bin/env python3
import sys, time, threading, rclpy
from rclpy.node import Node
from rclpy.qos import qos_profile_sensor_data
from geometry_msgs.msg import Twist, PoseStamped
from std_msgs.msg import String

ALT, GROUND = 1.05, 0.06
KP, VXY, VUP, VDN = 0.8, 0.6, 0.5, 0.4
OFFSETS = {   # offsets from the leader (x, y), per follower id
    'LINE':  {1: (-1.5, 0), 2: (-3.0, 0), 3: (-4.5, 0), 4: (-6.0, 0)},
    'ARROW': {1: (-1.5, 1.5), 2: (-1.5, -1.5), 3: (-3.0, 3.0), 4: (-3.0, -3.0)},
}
KEYS = {'1': 'LINE', '2': 'ARROW', 'T': 'TAKEOFF', 'L': 'LAND', 'LINE': 'LINE',
'ARROW': 'ARROW', 'TAKEOFF': 'TAKEOFF', 'LAND': 'LAND'}

def clip(v, lim): return max(-lim, min(lim, v))

class Base(Node):
    def __init__(self, idx, name):
        super().__init__(name)
        self.idx, self.pos = idx, None
        self.create_subscription(PoseStamped, f'/drone{idx}/ground_truth/pose',
                                 self._pose, qos_profile_sensor_data)
        self.pub = self.create_publisher(Twist, f'/gz/drone{idx}/cmd_vel', 10)
    def _pose(self, m):
        p = m.pose.position; self.pos = (p.x, p.y, p.z)
    def drive(self, t):
        x, y, z = self.pos
        ex, ey, ez = t[0]-x, t[1]-y, t[2]-z
        m = Twist()
        m.linear.x = clip(KP*ex, VXY); m.linear.y = clip(KP*ey, VXY)
        m.linear.z = clip(KP*ez*2, VUP if ez > 0 else VDN)
        self.pub.publish(m)
        return max(abs(ex), abs(ey), abs(ez))
```

```
class Leader(Base):
    def __init__(self):
        super().__init__(0, 'leader_drone0')
        self.cmd_pub = self.create_publisher(String, '/formation/command', 10)
        self.lp_pub = self.create_publisher(PoseStamped, '/formation/leader_pose',
10)
        self.target = None
        self.create_timer(0.05, self.loop)
    def loop(self):
        if self.pos is None: return
        m = PoseStamped(); m.header.frame_id = 'earth'
        m.pose.position.x, m.pose.position.y, m.pose.position.z = self.pos
        self.lp_pub.publish(m)
        if self.target: self.drive(self.target)
    def send(self, cmd):
        self.cmd_pub.publish(String(data=cmd))
        x, y, _ = self.target if self.target else self.pos
        if cmd == 'TAKEOFF': self.target = (self.pos[0], self.pos[1], ALT)
        if cmd == 'LAND': self.target = (x, y, GROUND)
        print(f'[LEADER] broadcast {cmd}')

class Follower(Base):
    def __init__(self, idx):
        super().__init__(idx, f'follower_drone{idx}')
        self.mode, self.fixed, self.leader, self.last = 'IDLE', None, None, 0
        self.create_subscription(String, '/formation/command', self.on_cmd, 10)
        self.create_subscription(PoseStamped,'/formation/leader_pose'
        ,self.on_leader, 10)
        self.create_timer(0.05, self.loop)
    def on_leader(self, m):
        p = m.pose.position; self.leader = (p.x, p.y, p.z)
    def on_cmd(self, m):
        cmd = m.data
        self.get_logger().info(f'received command from leader: {cmd}')
        if self.pos is None: return
        if cmd == 'TAKEOFF': self.mode, self.fixed = 'HOLD', (self.pos[0]
        ,self.pos[1], ALT)
        elif cmd == 'LAND':  self.mode, self.fixed = 'HOLD', (self.pos[0]
        ,self.pos[1], GROUND)
        elif cmd in OFFSETS: self.mode = cmd
    def loop(self):
        if self.pos is None or self.mode == 'IDLE': return
        if self.mode == 'HOLD': t = self.fixed
        elif self.leader is None: return
        else:
            ox, oy = OFFSETS[self.mode][self.idx]
            t = (self.leader[0]+ox, self.leader[1]+oy, ALT)
        err = self.drive(t)
        if time.time() - self.last > 3:
            self.last = time.time()
            self.get_logger().info(f'mode={self.mode} error={err:.2f} m')

def main():
    rclpy.init()
```

```
    if sys.argv[1] == 'follower':
        rclpy.spin(Follower(int(sys.argv[2]))); return
    node = Leader()
    threading.Thread(target=rclpy.spin, args=(node,), daemon=True).start()
    while node.pos is None: time.sleep(0.2)
    print('Leader commands: 1=LINE  2=ARROW  T=TAKEOFF  L=LAND  g dx dy=move leader
q=quit')
    while True:
        c = input('Leader command > ').strip()
        if c.lower() == 'q': break
        if c.upper() in KEYS: node.send(KEYS[c.upper()])
        elif c.startswith('g '):
            try:
                dx, dy = map(float, c.split()[1:3])
            except ValueError:
                print('usage: g <dx> <dy>, e.g. g 2 0'); continue
            x, y, z = node.target or node.pos
            node.target = (x+dx, y+dy, z); print(f'[LEADER] moving by ({dx}, {dy})')
    rclpy.shutdown()
main()
```

Appendix 4 – run_leader_follower.sh

Complete source listing reproduced from the supplied project archive. No source-code lines have been intentionally omitted.

```
#!/bin/bash
source /opt/ros/humble/setup.bash
source ~/aerostack2_ws/install/setup.bash
export
FASTRTPS_DEFAULT_PROFILES_FILE="$HOME/project_gazebo/config/fastdds_udp_only.xml"
export
FASTDDS_DEFAULT_PROFILES_FILE="$HOME/project_gazebo/config/fastdds_udp_only.xml"
cd ~/project_gazebo
for i in 1 2 3 4; do python3 leader_follower_dds.py follower $i > /tmp/follower$i.log
2>&1 & done
trap 'kill $(jobs -p) 2>/dev/null' EXIT
sleep 3
python3 leader_follower_dds.py leader
```

Appendix 5 – leader_tasks.py

Complete source listing reproduced from the supplied project archive. No source-code lines have been intentionally omitted.

```
#!/usr/bin/env python3
"""Leader assigns tasks, followers execute and report.
Usage: leader_tasks.py leader | follower <1-4>"""
import sys, time, json, threading, rclpy
from rclpy.node import Node
from rclpy.qos import qos_profile_sensor_data
from geometry_msgs.msg import Twist, PoseStamped
from std_msgs.msg import String
```

```
ALT, GROUND = 1.05, 0.06
KP, VXY, VUP, VDN, TOL = 0.8, 0.6, 0.5, 0.4, 0.2
OFFSETS = {
    'LINE':  {1: (-1.5, 0), 2: (-3.0, 0), 3: (-4.5, 0), 4: (-6.0, 0)},
    'ARROW': {1: (-1.5, 1.5), 2: (-1.5, -1.5), 3: (-3.0, 3.0), 4: (-3.0, -3.0)},
}
AREAS = {
    'A': [(-2.5, -2.0), (-2.5, 2.0), (-1.0, 2.0), (-1.0, -2.0)],
    'B': [(-0.5, -2.0), (-0.5, 2.0), (0.5, 2.0), (0.5, -2.0)],
    'C': [(1.0, -2.0), (1.0, 2.0), (2.5, 2.0), (2.5, -2.0)],
}
HELP = ('Leader commands: T=takeoff L=land 1=LINE 2=ARROW | mission [delay] | '
        'assign N HOLD|LAND|LINE|ARROW|GOTO x y z | g dx dy | status | q')

def clip(v, lim): return max(-lim, min(lim, v))
def alt_of(d): return ALT + 0.3 * (d - 1)      # separate altitude per drone
def wps(area, z): return [[x, y, z] for x, y in AREAS[area]]

class Base(Node):
    def __init__(self, idx, name):
        super().__init__(name)
        self.idx, self.pos = idx, None
        self.create_subscription(PoseStamped, f'/drone{idx}/ground_truth/pose',
                                 self._pose, qos_profile_sensor_data)
        self.pub = self.create_publisher(Twist, f'/gz/drone{idx}/cmd_vel', 10)
    def _pose(self, m):
        p = m.pose.position; self.pos = (p.x, p.y, p.z)
    def drive(self, t):
        x, y, z = self.pos
        ex, ey, ez = t[0]-x, t[1]-y, t[2]-z
        m = Twist()
        m.linear.x = clip(KP*ex, VXY); m.linear.y = clip(KP*ey, VXY)
        m.linear.z = clip(KP*ez*2, VUP if ez > 0 else VDN)
        self.pub.publish(m)
        return max(abs(ex), abs(ey), abs(ez))

class Leader(Base):
    def __init__(self):
        super().__init__(0, 'leader_drone0')
        self.lock = threading.Lock()
        self.lp_pub = self.create_publisher(PoseStamped, '/formation/leader_pose',
10)
        self.task_pub = {d: self.create_publisher(String,
f'/formation/task/drone{d}', 10)
                         for d in (1, 2, 3, 4)}
        self.create_subscription(String, '/formation/status', self.on_status, 10)
        self.target, self.busy, self.done, self.owner = None, set(), set(), {}
        self.create_timer(0.05, self.loop)

    def loop(self):
        if self.pos is None: return
        m = PoseStamped(); m.header.frame_id = 'earth'
        m.pose.position.x, m.pose.position.y, m.pose.position.z = self.pos
        self.lp_pub.publish(m)
        if self.target: self.drive(self.target)
```

```
    def send_task(self, d, task):
        self.task_pub[d].publish(String(data=json.dumps(task)))
        print(f'[LEADER] -> drone{d}: {task["type"]}' +
              (f' area {task["area"]} waypoints {[i+1 for i in task["todo"]]}'
               if task['type'] == 'SURVEY' else ''))

    def broadcast(self, what):
        x, y, z = self.target or self.pos
        if what == 'TAKEOFF': self.target = (self.pos[0], self.pos[1], ALT)
        if what == 'LAND': self.target = (x, y, GROUND)
        task = {'type': 'FORMATION', 'name': what} if what in OFFSETS else {'type':
what}
        with self.lock: self.busy.clear()
        for d in (1, 2, 3, 4): self.send_task(d, task)

    def start_mission(self, delay=False):
        with self.lock:
            self.done.clear(); self.owner = {}
            for d, area in ((1, 'A'), (2, 'B'), (3, 'C')):
                t = {'type': 'SURVEY', 'area': area, 'wps': wps(area, alt_of(d)),
                     'todo': [0, 1, 2, 3]}
                if delay and d == 1: t['delay_at'] = 2      # simulated delay at A3
                self.busy.add(d); self.owner[area] = d
                self.send_task(d, t)

    def assign_cmd(self, a):
        d, what = int(a[0]), a[1].upper()
        if what in ('HOLD', 'LAND'): t = {'type': what}
        elif what in OFFSETS: t = {'type': 'FORMATION', 'name': what}
        elif what == 'GOTO': t = {'type': 'GOTO', 'target': [float(v) for v in
a[2:5]]}
        else: print('unknown task'); return
        with self.lock: self.busy.discard(d)
        self.send_task(d, t)

    def on_status(self, m):
        s = json.loads(m.data); d, ev = s['drone'], s['event']
        if ev == 'WP_DONE':
            with self.lock: self.done.add((s['area'], s['index']))
            print(f'[LEADER] drone{d} finished {s["area"]}{s["index"]+1}')
        elif ev == 'DELAYED':
            print(f'[LEADER] drone{d} reports DELAY, remaining {s["area"]} '
                  f'waypoints {[i+1 for i in s["remaining"]]}')
            self.reassign(d, s['area'], s['remaining'])
        elif ev == 'TASK_DONE':
            with self.lock: self.busy.discard(d)
            print(f'[LEADER] drone{d} finished its task')
            self.verify()
        elif ev == 'GOTO_DONE':
            print(f'[LEADER] drone{d} reached its GOTO point')

    def reassign(self, src, area, remaining):
        with self.lock:
            self.busy.discard(src)
```

```
            helpers = [d for d in (1, 2, 3, 4) if d not in self.busy and d != src]
            if not helpers:
                print('[LEADER] ERROR: no idle helper available'); return
            h = helpers[-1]                    # drone4 is the spare
            self.busy.add(h)
        print(f'[LEADER] TRANSFER: drone{src} -> drone{h}, area {area}')
        self.send_task(h, {'type': 'SURVEY', 'area': area, 'wps': wps(area,
alt_of(h)),
                           'todo': list(remaining)})
        self.send_task(src, {'type': 'HOLD'})

    def verify(self):
        with self.lock:
            ok = all((a, i) in self.done for a in AREAS for i in range(4))
            busy = bool(self.busy)
        if ok and not busy:
            print('[LEADER] MISSION VERIFIED: areas A, B, C 4/4 each, assigned by
leader')

    def print_status(self):
        with self.lock:
            for a in AREAS:
                n = sum(1 for (ar, _) in self.done if ar == a)
                print(f'  Area {a}: {n}/4')
            print(f'  busy drones: {sorted(self.busy)}')

class Follower(Base):
    def __init__(self, idx):
        super().__init__(idx, f'follower_drone{idx}')
        self.task, self.leader = None, None
        self.st_pub = self.create_publisher(String, '/formation/status', 10)
        self.create_subscription(String, f'/formation/task/drone{idx}', self.on_task,
10)
        self.create_subscription(PoseStamped, '/formation/leader_pose',
self.on_leader, 10)
        self.create_timer(0.05, self.loop)
    def status(self, event, **kw):
        kw.update(drone=self.idx, event=event)
        self.st_pub.publish(String(data=json.dumps(kw)))
    def on_leader(self, m):
        p = m.pose.position; self.leader = (p.x, p.y, p.z)
    def on_task(self, m):
        if self.pos is None: return
        d = json.loads(m.data); k = d['type']
        if k in ('HOLD', 'LAND', 'TAKEOFF'):
            x, y, z = self.pos
            d['target'] = [x, y, {'HOLD': z, 'LAND': GROUND, 'TAKEOFF': ALT}[k]]
        self.task = d
        self.get_logger().info(f'task from leader: {k} ' + str({a: d[a] for a in
('area', 'name', 'todo') if a in d}))
    def hold_here(self):
        self.task = {'type': 'HOLD', 'target': list(self.pos)}
    def loop(self):
        if self.pos is None or self.task is None: return
        t = self.task; k = t['type']
```

```
        if k in ('HOLD', 'LAND', 'TAKEOFF'):
            self.drive(t['target'])
        elif k == 'GOTO':
            if self.drive(t['target']) < TOL:
                self.status('GOTO_DONE'); self.hold_here()
        elif k == 'FORMATION':
            if self.leader is None: return
            ox, oy = OFFSETS[t['name']][self.idx]
            self.drive((self.leader[0]+ox, self.leader[1]+oy, ALT))
        elif k == 'SURVEY':
            if not t['todo']:
                self.status('TASK_DONE', area=t['area']); self.hold_here(); return
            i = t['todo'][0]
            if t.get('delay_at') == i and not t.get('delayed'):
                t['delayed'] = True
                self.status('DELAYED', area=t['area'], remaining=list(t['todo']))
                self.hold_here(); return
            if self.drive(t['wps'][i]) < TOL:
                self.status('WP_DONE', area=t['area'], index=i)
                t['todo'].pop(0)

def main():
    rclpy.init()
    if sys.argv[1] == 'follower':
        rclpy.spin(Follower(int(sys.argv[2]))); return
    L = Leader()
    threading.Thread(target=rclpy.spin, args=(L,), daemon=True).start()
    while L.pos is None: time.sleep(0.2)
    print(HELP)
    while True:
        try: c = input('Leader command > ').strip()
        except EOFError: break
        if not c: continue
        a = c.split(); k = a[0].upper()
        try:
            if k == 'Q': break
            elif k in ('T', 'L', '1', '2'):
                L.broadcast({'T': 'TAKEOFF', 'L': 'LAND', '1': 'LINE', '2':
'ARROW'}[k])
            elif k == 'MISSION': L.start_mission(len(a) > 1 and a[1].lower() ==
'delay')
            elif k == 'ASSIGN': L.assign_cmd(a[1:])
            elif k == 'G':
                x, y, z = L.target or L.pos
                L.target = (x+float(a[1]), y+float(a[2]), z)
            elif k == 'STATUS': L.print_status()
            else: print(HELP)
        except Exception as e:
            print('error:', e, '\n' + HELP)
    rclpy.shutdown()
main()
```

Appendix 6 – run_leader_tasks.sh

Complete source listing reproduced from the supplied project archive. No source-code lines have been intentionally omitted.

```
#!/bin/bash
source /opt/ros/humble/setup.bash
source ~/aerostack2_ws/install/setup.bash
export
FASTRTPS_DEFAULT_PROFILES_FILE="$HOME/project_gazebo/config/fastdds_udp_only.xml"
export
FASTDDS_DEFAULT_PROFILES_FILE="$HOME/project_gazebo/config/fastdds_udp_only.xml"
cd ~/project_gazebo
for i in 1 2 3 4; do python3 leader_tasks.py follower $i > /tmp/follower$i.log 2>&1 &
done
trap 'kill $(jobs -p) 2>/dev/null' EXIT
sleep 3
python3 leader_tasks.py leader
```